

Refresh Decision-Support for Declining Content

FlyRank ML Internship — Capstone Research Paper

September 2026

A ranked scoring engine for "which page do I open first?" — built on real, anonymized search data, validated with whole-client holdout, and shipped as a human review queue rather than an autopilot.

Abstract

Question — which existing pages should a search team open first in a content-refresh review?

Method — we build a leakage-safe random-forest ranking of decline risk alongside a transparent hand rule, and validate both with cross-validation that holds whole pseudonymous clients out of every training fold.

Result — on that honest split the model reaches $\text{Precision@50} = 0.73$ with $\text{ROC-AUC} = 0.65$ against a 54% base rate, beating the hand rule (0.61) on the same folds.

Validation — a naive row-level split would have advertised an optimistic 0.94 Precision@50 ; the drop to 0.73 is the measured cost of letting pages from the same client leak across folds.

Outcome — the deliverable is a ranked refresh queue with transparent reason codes that supports human editorial review — it observes, measures, and orders; it never publishes, deletes, or promises rank recovery.

1 Introduction & problem statement

Every week a content or SEO reviewer has the same budget-limited decision: **which page do I open first?** Content ages, ranks slip, click-through erodes, and most of it is invisible until someone looks. This work supports that decision — it orders the thousands of pages a portfolio serves so that a human starts where the evidence is loudest.

- **Unit of analysis** — one row is one page/content in a client portfolio (30,000 pages across 32 pseudonymous clients).
- **Output** — a ranked refresh-opportunity queue: rank, action, reason codes, and a confidence bucket per page.
- **Who acts** — a human reviewer. The queue decides the order to look, not the content of the edit.
- **Cost of a wrong call** — refreshing a healthy page wastes editorial effort and can disturb a page that works; missing a genuinely declining page lets visibility erode silently.

Why does data help at all? The signals that mark an opportunity — visibility, trend state, engagement, content length, age — interact non-linearly, and the first honest attempt at a fixed rule ($\text{stale} \times \text{visible} \rightarrow \text{inspect clicks/engagement}$) surfaced only **17 actionable pages out of 30,000**. An if-statement is too brittle for a decision this noisy; a machine-learned ranker is worth trying precisely because it can weigh many proxies at once. None of this claims prediction of Google's ranking — the goal is *observed* trend state, *directional* risk, and *decision-support* for a human reviewer.

2 Data

Release & table. This work uses the starter release that ships with the internship repo — the single anonymized table `data/raw/content_refresh_anonymized.csv` (30,000 rows × 44 columns, 32 pseudonymous clients). It is the small bundled slice, not the full 79M-row warehouse, and it contains no client names, domains, URLs, titles, or keywords.

Time geometry. Pages are 90–564 days old; their last update is 1–373 days ago; impressions are summarized over a 90-day window. No calendar field survived anonymization, so no seasonality can be modelled and no forward-looking "do this and watch" claim is possible. The data is a single cross-sectional snapshot.

Fields and exclusions (public-safe). Fields were sorted into four buckets: features, label, context, excluded.

Bucket	Fields	Why
Label	<code>trend_direction</code>	The observed outcome (<code>down</code> = declining). Never a feature.
Excluded — label-derived	<code>trend_pct</code>	Label source; alone it separates decline with AUC = 1.000. Pure leakage.
Excluded — identifiers	<code>content_id</code> , <code>client_id</code>	Identity only. <code>client_id</code> is used <i>only</i> to group cross-validation, never as a signal.
Excluded — product flags	<code>provider_used</code> , <code>model_used</code>	Not model features per the data dictionary.
Handled	<code>avg_position</code> = 0	"0" means no data, not rank 0 — 1,205 rows are re-coded to missing.

Known gaps you read about before the model does. Missingness is systematic, not random: keyword fields (`search_volume`, `competition`, `cpc`) are missing in 100% of **feedly article** rows; `word_count`/`char_count` are missing on 7,699 rows; `avg_position` is missing/zero on 1,205 rows. Rows with thinner evidence are scored with capped confidence rather than silently excluded.

3 Methodology

Assumptions

- The observed trend state (`down`, `up`, `stable`, `flat`, `new`) is a reasonable proxy for "a human should look here".
- A single 90-day snapshot can *rank within* a portfolio, but cannot predict the future for a page.
- Features and label must never share a window — enforced, and measured (see leakage checks).
- Pseudonymous client groups are exchangeable-ish units, so holding whole clients out is the honest test of generalization.

Label

`is_declining` = `trend_direction` == "down". Base rate: **54.2%** (16,262 of 30,000 pages). Any precision figure near 0.54 is just the coin flip — every claim in §4 is read against this.

Features (leakage-safe, 47 columns)

13 numeric + 8 categorical signals, one-hot encoded. The decisive construction is `impressions_earliest_slice = max(0, impressions_90d - impressions_last_30d - impressions_prev_30d)` (log-transformed), which removes the two 30-day windows the label is computed from. The 90-day window overlaps the label's windows by a *measured* 0.562 of its volume — rebuilding it as the earliest slice is the crosstalk control. Missing numerics are filled with 0; missing categories become an explicit `unknown`; `avg_position = 0` is re-coded missing.

Baseline (the bar to beat)

A transparent hand rule, stated in plain words before code: *"if a page is stale and still visible, check its CTR and engagement to pick a refresh action."* Scored as `stale( $\geq 180$  d)  $\times$  visible( $\geq 500$  impressions)  $\times$  impressions_90d`. It is the fair comparison: same data, same metric, same folds as the model. In the Week-4 build it flagged only 17 non-monitor pages — its top-20 declining rate was 0.85.

Model

Random forest (300 trees, max depth 8, balanced class weights, seed 42) predicting decline probability. The shape of the task — many interacting proxies, heavy tails (impressions skew ≈ 11.4 , CTR ≈ 17.4) — matches an ensemble tree model better than a single hand rule.

Validation design

GroupKFold(5) on `client_id`: whole clients are held out of each training fold. A naive row-level split is *wrong here* because dozens of pages share a client and the split lets near-duplicate rows leak across folds — the measured price is shown in §4. No time-aware split is possible: no calendar survived anonymization.

Leakage checks

Each candidate feature was scored by a single-split decision tree against the label. `trend_pct` scores 1.000 — excluded. Every kept feature stays ≤ 0.62 ; `impressions_prev_30d` (0.623) is flagged as a watch-item and is never used raw. The label strings themselves never enter the feature vector.

4 Results — model vs. baseline on the same split

The central methodological result. The same leakage-safe model under two validation designs:

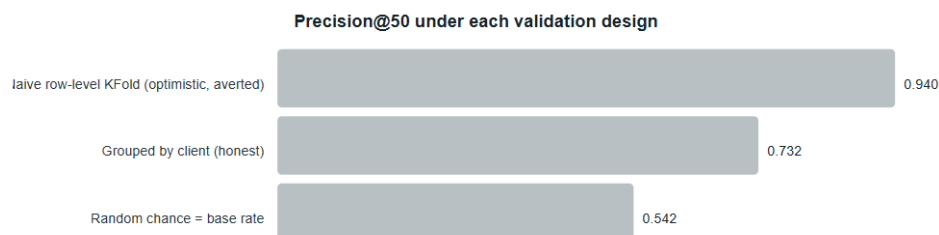


Figure 1: Fig. 1. Precision@50 by validation design. A naive row-level KFold (grey) reports 0.94 and would have been the headline; holding whole clients out (teal) lands at 0.73, against a 54.2% base rate (outline). The 0.21 gap is shared-client optimism, not signal.

The honest comparison. Model versus the Week-4 hand rule, measured on the identical client-grouped folds:

Metric (mean over 5 client-held-out folds)	Value
ROC-AUC	0.65
Precision@20	0.80
Precision@50	0.73
Declining rate at the top of the queue	0.91



Figure 2: Fig. 2. Precision@K, model vs. rule, same client-grouped folds. The model adds real but modest lift (+0.10 at P@20, +0.12 at P@50 over the rule). The bigger lesson is on the left of the table: the rule barely fires at all.

What the model actually reads. Top features on the leakage-safe vector are the earliest visible slice (`log_impressions_earliest_slice` 0.19), `avg_position` (0.15) and `content_age_days` (0.10). Consistent with the signal audit, staleness alone contributes little (`days_since_last_update` ≈ 0.03 ranking), which is exactly why a multi-signal model beats the single-window rule.

Where it errs. On the test split, false negatives are older (median content age 417 days) than false positives (165 days): the model over-relies on the observed age–decline association and under-flags older declining pages. The signal audit explains the trap — `content_age_days` vs. trend is statistically detectable (Kruskal $p \approx 0$) but small and *reversed* from intuition (newer content skews down more), so any rule leaning on "old = at risk" is on thin ice. The recommendation: treat older declining pages as a manual second look, not a model verdict.

Observed signals worth remembering (Week-4 signal audit). Position-vs-CTR correlates 0.36 (real but weak per page — the clean group curve overstates one page’s predictability); search volume vs. impressions ≈ 0 (0.03); staleness vs. decline is statistically tiny (rank-biserial 0.055) — a content team should not prioritize refresh by staleness alone.

5 Limitations & honest framing

- **Cross-sectional snapshot.** One 90-day window, no calendar. This ranks a portfolio; it cannot say a refresh *will* recover a page — no intervention, no experiment, no causal claim.
- **Label-adjacent information.** The playbook’s reason codes include the *observed* trend state, and the baseline rule reads the same impression windows the label uses. That is legitimate for decision-support (a reviewer can see the same line), but it means queue precision numbers are not a forward prediction.

- **Unbalanced evidence.** No-position rows (1,205), no-keyword rows (2,468), and missing content fields (7,699) get partial reason codes and capped confidence; missingness is systematic, not random.
- **Uneven clients.** 32 clients, wildly uneven volume — one client supplies 96 of the top-100 queue rows. Per-client performance varies (± 0.18 P@50), so single-client wins or losses should not be over-read.
- **Not a claim about Google.** This ranks pages inside one pseudonymized portfolio. Nothing here proves or predicts Google’s algorithm.
- **Wording discipline.** observed / measured / directional / decision-support — every recommendation in §6 is framed that way.

6 Ranked recommendations — the action playbook

The shipped output is a ranked queue (`work/outputs/playbook_refresh_queue.csv`): every page carries a final score, a suggested action, and human-readable reason codes. A reviewer works top-down; `monitor` is the explicit "watch, don't touch" pile and is not a safety guarantee.

#	Action	Plain meaning
1	<code>refresh_and_review_ctr</code>	Declining, still visible near page 1, low CTR — review title / meta / intent for the query it is losing.
2	<code>refresh_and_review_engagement</code>	Declining, visible, low engagement — review body / UX against what the query wants.
3	<code>refresh</code>	Declining or at-risk with no single confirmed lever — general refresh, re-check next window.
4	<code>expand_and_refresh</code>	Thin but visible pages with demand — need depth, not a copy edit.
5	<code>monitor</code>	No confirmed lever yet.

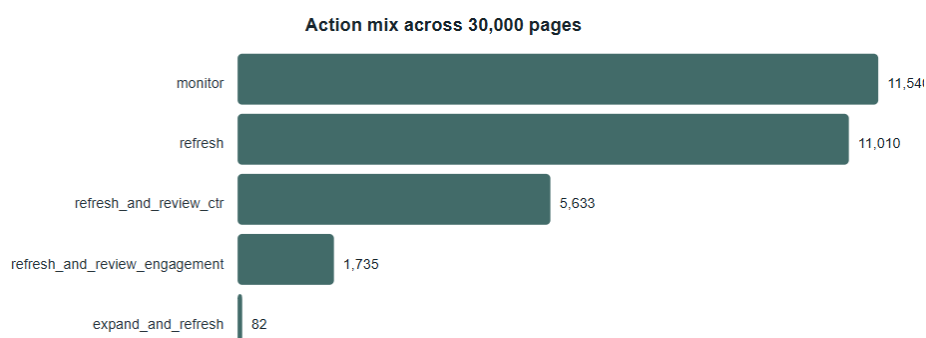


Figure 3: Fig. 3. Action mix across all 30,000 pages. The queue is deliberately conservative — more than a third of pages wait on monitor.

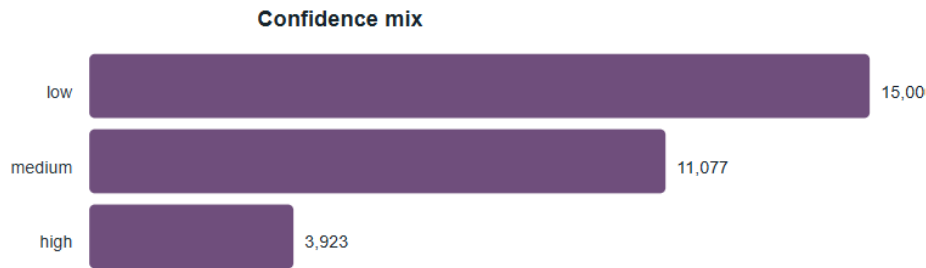


Figure 4: Fig. 4. Confidence mix. High-confidence rows are the safe first production use: manually verify, compare against editorial context, then act.

Reason codes a reviewer sees

Confidence, so the reviewer can triage

What a human must check before acting

1. Is the trend real? Two 30-day windows can't see a launch, a sitemap break, or a tracking change.
2. Is the reason code the actual problem? A low-CTR flag on page 1 can be a SERP feature stealing the click, not a bad title.
3. Is the measurement intact? A broken GA4 tag makes "low engagement" noise.
4. Was the last update logged? `stale_visible_page` trusts `days_since_last_update` to be truthful.
5. Did the model have the evidence? Missing position / keyword data drop confidence a notch.

Whole-queue, **1,461 rows** carry model risk ≥ 0.65 while the observed trend is *not* down (1,025 of them on visible pages) — these are the flagged "model says look, label says calm" candidates a reviewer should inspect first before trusting placement.

No-go list — never automated. No automatic publishing or unpublishing; no deletes or deprecations from a score; no acting on `declining_with_demand` as if we could flip it; no scoring brand-new pages (<90 days), no-position rows, or clients without GA4-style data; and never a claim that the model "predicted Google's algorithm".

When the recommendations go stale

Re-score every 4 weeks or on client data resync. Retrain when honest P@50 falls below 0.60 or the top-50 declining rate falls within +0.10 of base rate. Raise an alert when a single client holds more than a third of the top-100 — *today this check fails* (96 of the top-100 rows are one client), which is a live reason to review per-client caps before leaning on the queue.

7 Reproducibility

Everything runs from the public repo — each notebook executes top-to-bottom against the bundled starter data and writes its artifacts to `work/outputs/`.

Environment. Python 3.13 with pandas 2.2.3, numpy 2.4.4, scikit-learn 1.8.0 (`requirements.txt`).
Seeds: `random_state=42` for the random forest and every split. Re-run with:

Piece	Where
Repo	github.com/TaherBenAfia/Fly2
Problem framing	work/notebooks/w01_research_question.ipynb · w02_ml_task_framing.ipynb
Data contract & leakage	work/notebooks/w03_data_contract.ipynb · w03_feature_leakage_check.ipynb
Baseline & signal audit	work/notebooks/w04_baseline_score.ipynb · w04_signal_audit.ipynb
Model	work/notebooks/w05_model.ipynb
Validation audit (grouped split)	work/notebooks/w06_validation_audit.ipynb
Ranked action playbook + exports	work/notebooks/w07_action_playbook.ipynb → work/outputs/playbook_refresh_queue.csv , playbook_metrics.json

```
pip install -r requirements.txt · jupyter nbconvert -to notebook -execute -inplace
work/notebooks/w07_action_playbook.ipynb
```

Or open any notebook in Google Colab with the bundled "Open in Colab" badge. All numbers in this paper match a fresh run of these notebooks against the pinned data file.

8 Acknowledgments & data credit

Built on the FlyRank ML Internship dataset — flyrank.ai — the anonymized, public-safe slice of real search and content data that made a capstone like this possible without exposing anyone’s private portfolio.

Thanks to the internship program and its track leads for the framework — documents, starter repo, and the discipline of honest framing (observed / measured / directional / decision-support) that shaped every number on this page. The research method, validation design, and all analysis are the author’s own.